Centurion

UNIVERSITY

Shaping Lives...

Empowering Communities...

Smart Feedback System for CUTM Hostel Mess

(FinTech Fraud Detection)

PROJECT REPORT

Submitted by

P.TATTVAMASI(230301120097)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMUTER SCIENCE AND ENGINEERING

SCHOOLOFENGINEERINGANDTECHNOLOGY BHUBANESWAR CAMPUS

CENTURIONUNIVERSITYOFTECHNOLOGYANDMANAGEMENT ODISHA

APRIL 2026

DEPARTMENT OF COMPUTER SCIENCE SCHOOL OF ENGINEERING AND TECHNOLOGY
BHUBANESWAR CAMPUS

BONAFIDE CERTIFICATE

Certified that this project report of **"FinTech fraud detection"** is the Bonafide work of **"P.TATTVAMASI"** who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

SIGNATURE

(Mr.Sushanta Meher)
(Guided By)

Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university.

SIGNATURE

(Dr.Rahul Roy)
HEAD OF THE DEPARTMENT / DEAN OF THE SCHOOL
Professor of Computer Science and Engineering

DEPARTMENT SEAL

DECLARATION

I hereby declare that the project entitled **"FinTech fraud detection"** submitted for the **"Minor Project"** of 7th semester B. Tech in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

Name of the Student: P.TATTVAMASI

Signature of the Student: _____

Registration No: 230301120097

Place: Bhubaneswar **Date:** April 2026

ACKNOWLEDGEMENTS

I wish to express my profound and sincere gratitude to Prof. Dr.Rahul Roy, Department of Computer Science Engineering, SoET, Bhubaneswar Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank Prof. Dr.Rahul Roy, Head of the Dept. of Department of Computer Science and Engineering, SoET, Bhubaneswar Campus and Prof. Dr.Swarnaprabha Jena, Dean, School of Engineering and Technology, Bhubaneswar Campus for extending their support during Course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered during various stages of image interpretation by Mr.Sushanta Meher. I am highly grateful to parents who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to my parents for their constant encouragement, cooperation, and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavor.

Name of the Student: P.TATTVAMASI

Signature of the Student: _____

Registration No: 230301120097

Place: Bhubaneswar **Date:** April 2026

TABLE OF CONTENTS

Chapter	Title	Page
	Certificate	ii
	Declaration	iii
	Acknowledgement	iv
Chapter 1	**Abstract**	1
Chapter 2	**Introduction**	2
	2.1 Background and Motivation	2
	2.2 Problems in Current Fraud Detection Systems	3
	2.3 Need for a Graph-Based & Adaptive Solution	3
	2.4 Objectives of the Project	4
Chapter 3	**Literature Review**	5
	3.1 Existing AML & Rule-Based Monitoring	5
	3.2 Graph Analytics & Multi-Hop Layering Motifs	6
	3.3 Concept Drift & Model Epistemic Uncertainty	6
	3.4 Limitations of Traditional Systems and Gap Analysis	7
Chapter 4	**System Requirements**	8
	4.1 Overview & Hardware Environment	8
	4.2 Core Software Technologies (Java 17, Spring Boot, React, Vite)	8
	4.3 System Configuration Requirements Table	10
Chapter 5	**System Architecture**	11
	5.1 Architectural Overview (Multi-Tier Enterprise Design)	11
	5.2 Component Architecture & In-Memory Pipeline	12
	5.3 Data Flow & System Flow Diagram	13
Chapter 6	**Working Principle**	14
	6.1 Real-Time Streaming Ingestion Pipeline	14
	6.2 Graph Topology Engine (Circular Layering DFS & Mule Hubs)	14
	6.3 Dynamic Spend Z-Score & Velocity Deviation	15
	6.4 Epistemic Uncertainty & Active Learning Prioritization	15
	6.5 Explainable AI (XAI) Attribution Waterfall	16
Chapter 7	**Implementation**	17
	Design & Prototyping	17
	Frontend Structure	17
	Core Components	18
	Data Handling	19
Chapter 8	**Algorithms / Logic**	20
	8.1 Depth-Bounded DFS Cycle Detection Algorithm	20

Chapter	Title	Page
	8.2 Mule Network Accumulator & Disperser Logic	21
	8.3 Epistemic Uncertainty & Active Learning Formulas	22
Chapter 9	**System Validation and Observations**	23
Chapter 10	**System Output / Screenshots**	24
	10.1 Executive KPI Command Center Dashboard	24
	10.2 Real-Time Transaction Ingestion Stream	25
	10.3 Directed Fraud Graph Topology Visualizer	26
	10.4 Explainable AI (XAI) Investigation Modal	27
	10.5 Attack Simulation & Concept Drift Workbench	28
Conclusion	**Conclusion & Future Scope**	29
References	**Academic References**	30
Assessment	**Internal Assessment Rubrics & CO Attainment**	31

CHAPTER 1: ABSTRACT

In contemporary digital banking and instant payment infrastructures, financial crime syndicates have evolved far beyond simplistic, single-account fraud techniques. Modern money laundering operations employ sophisticated money mule networks, structured circular layering cycles, and high-frequency micro-smurfing to disperse illicit funds while evading conventional transaction-monitoring thresholds. Compounding this challenge is the phenomenon of *adversarial concept drift*, wherein criminal typologies dynamically shift across payment channels, geographical regions, and velocity patterns to evade stationary supervised machine learning models. Furthermore, traditional rule-based compliance engines generate over 90% false positive rates, creating severe alert fatigue and delaying critical asset freezes.

In this project, we developed **FraudGraph AI** — an adaptive financial transaction-monitoring and fraud intelligence platform specifically engineered to address coordinated mule rings and temporal concept drift. The system was implemented using **Java 17** and **Spring Boot 3.3** on the backend, coupled with a high-density, real-time **React 18 / Tailwind CSS / Recharts** compliance workstation on the frontend.

The platform integrates four core detection engines: (1) a graph topology engine utilizing an in-memory sliding-window directed multigraph with depth-bounded Depth-First Search (depth ≤ 4) to detect cyclic layering loops ($A \rightarrow B \rightarrow C \rightarrow A$) and mule hubs; (2) a behavioral spend profiling engine computing 10-minute velocity counters and spend Z-score deviations; (3) an adaptive concept drift detector tracking streaming distribution distance; and (4) an epistemic uncertainty estimator calculating decision-boundary ambiguity (U in [0.02, 0.98]). High-uncertainty, high-risk cases are mathematically prioritized for human compliance officers via an active-learning triage formula ($0.60 * \text{Risk} + 40.0 * U$).

Experimental validation over a benchmark dataset of 5,000 synthetic transactions (3.4% fraud rate) demonstrated an average sub-second processing latency of **14.5 ms** in Spring Boot. Across a temporal concept drift boundary, the mean model uncertainty exhibited a statistically significant jump from **0.4346** to **0.5384 (+0.1038 shift)**, proving that the model successfully flags novel evasion vectors for human investigation while providing deterministic Explainable AI (XAI) factor breakdowns in compliance with regulatory standards.

CHAPTER 2: INTRODUCTION

Background and Motivation

The rapid proliferation of digital financial rails, real-time wire transfers, and Unified Payments Interfaces (UPI) has made sub-second financial settlement a cornerstone of the modern global economy. However, the elimination of clearing delays has also dramatically empowered financial criminals. Fraudulent transactions execute and settle irreversibly within milliseconds, rendering traditional post-facto audit mechanisms completely obsolete.

According to the United Nations Office on Drugs and Crime (UNODC), up to $2 trillion in illicit funds is laundered globally each year. Rather than executing large, conspicuous transfers, organized money laundering syndicates employ "smurfing" and decentralized money mule rings. Criminals distribute illicit proceeds into dozens of small transfers across intermediary shell accounts, routing funds in circular or bipartite structures before re-aggregating the capital. In such environments, isolated account-level evaluations fail entirely, necessitating real-time relationship graph analytics.

Problems in Current Fraud Detection Systems

Traditional Anti-Money Laundering (AML) transaction monitoring systems suffer from four critical structural flaws:

1. **Rigid Rule Thresholds:** Legacy systems rely on static thresholds (e.g. transfers > $10,000). Criminals trivially evade these rules by splitting transactions into values just below regulatory limits (structuring).
2. **Node-Centric Isolation:** Relational databases store transactions as independent tabular records, blind to multi-hop topological structures such as circular layering loops (A → B → C → A).
3. **Adversarial Concept Drift:** Supervised machine learning classifiers trained on stationary historical distributions experience catastrophic decay when fraudsters adapt channels, payment amounts, and velocities.
4. **Investigator Alert Fatigue:** Over 90% of alerts generated by traditional rule engines represent false positives, burying genuine criminal networks under an unmanageable queue of benign transactions.

Need for a Graph-Based & Adaptive Solution

To overcome these challenges, a modern FinTech fraud detection platform must treat financial operations as a continuous, dynamic directed graph. By examining the relationships between accounts within sliding temporal windows, graph analytics can detect circular layering loops and accumulator mule clusters in real time.

Furthermore, the system must quantify *model epistemic uncertainty*. In situations where data patterns diverge due to concept drift or contradictory signals (e.g. benign spend but circular routing), the system must elevate ambiguous, high-risk cases to human compliance investigators. This forms a closed-loop **Active Learning** paradigm that continuously strengthens the model.

Objectives of the Project

1. To build a real-time transaction ingestion and scoring engine capable of sub-20ms decisions using Java 17 and Spring Boot 3.3.
2. To implement an in-memory sliding-window directed multigraph with depth-bounded DFS cycle detection (depth ≤ 4) and degree-based mule network identification.
3. To formulate an epistemic uncertainty calculus fusing boundary margin, signal dissonance, and sliding-window concept drift distance.
4. To construct an active-learning case prioritization algorithm (PriorityScore = 0.60 * Risk + 40.0 * U).

5. To develop a high-density React 18 compliance workstation featuring interactive SVG graph visualization, live ingestion feeds, and Recharts KPI analytics.
6. To deliver transparent Explainable AI (XAI) factor waterfall breakdowns for full regulatory auditability.

CHAPTER 3: LITERATURE REVIEW

Existing Anti-Money Laundering (AML) & Static Rule Systems

The early foundational literature in financial crime detection centered on rule-based expert systems (Bolton and Hand, 2002). These systems codified institutional compliance guidelines into static deterministic filters. While effective at enforcing mandatory legal reporting thresholds (such as the US Bank Secrecy Act CTR limit of $10,000), Phua et al. (2010) demonstrated that rule-based systems suffer from extreme rigidity and are easily bypassed by organized syndicates using structuring techniques.

Graph Analytics in Financial Anomaly Detection

Network and graph-based analysis has emerged as the premier mechanism for detecting multi-entity collusion. Savage et al. (2016) demonstrated that financial crime typologies naturally manifest as specific topological subgraphs, including circular directed cycles, dense bipartite networks, and high-degree star hubs. However, standard graph algorithms (such as Tarjan's strongly connected components or full-network PageRank) exhibit high polynomial complexity, making them computationally impractical for sub-second streaming pipelines. Kumar and Shah (2018) showed that depth-bounded local graph traversals (depth ≤ 4) within sliding windows achieve linear $O(V + E)$ complexity while successfully capturing smurfing and layering cycles.

Concept Drift & Model Uncertainty in Streaming Data

In streaming transactional environments, data distributions constantly shift—a phenomenon formalized by Gama et al. (2014) as concept drift. When fraudsters shift from high-value wire transfers to high-frequency mobile payments, stationary classifiers experience severe accuracy degradation. Bifet and Gavaldà (2007) introduced adaptive sliding-window variance tracking (ADWIN) to detect non-stationary stream behavior.

In high-stakes compliance domains, model uncertainty quantification is essential. Hüllermeier and Waegeman (2021) formalized the separation between aleatoric uncertainty (stochastic noise) and epistemic uncertainty (model lack of knowledge). In active learning frameworks (Settles, 2012), uncertainty sampling prioritizes decision-boundary cases, enabling targeted human feedback. FraudGraph AI builds upon this foundation by coupling epistemic uncertainty with streaming concept drift.

Limitations of Traditional Systems and Gap Analysis

Capability	Traditional Rule Systems	Standard ML Classifiers	FraudGraph AI (Proposed)
Relationship Mapping	None (Isolated accounts)	Basic aggregated features	**Sliding-window directed multigraph with DFS cycle detection**
Concept Drift Adaptation	Zero (Manual re-thresholding)	Fails on distribution shifts	**Statistical distance tracking & epistemic drift scaling**
Investigation Triage	First-In, First-Out (FIFO)	Naive probability score	**Dual-axis ranking: 0.6 * Risk + 40.0 * Uncertainty**
Explainability (XAI)	Rule text only	Black-box numeric score	**Deterministic factor attribution JSON waterfall**

CHAPTER 4: SYSTEM REQUIREMENTS

Overview

FraudGraph AI is engineered as an enterprise-grade multi-tier web application. The platform is designed with zero-setup defaults for instant local evaluation using in-memory databases and embedded web containers, while fully supporting enterprise production deployment with MySQL 8 and containerized microservices.

Development Environment

The backend requires a modern Java Development Kit (JDK 17 LTS or JDK 25) with Apache Maven 3.8+ for dependency management and automated testing. The frontend requires a Node.js runtime (v18.0 or higher) with npm. Visual Studio Code and IntelliJ IDEA serve as the recommended IDEs.

Core Software Technologies

- **Java 17 / 25 LTS:** Core backend programming language, chosen for high-throughput concurrent collections, strong type safety, and memory management.
- **Spring Boot 3.3.4:** Application framework providing dependency injection, embedded Tomcat web server, RESTful web services, and scheduled streaming tasks.
- **Spring Security 6.3 & JJWT 0.12:** Stateless security gateway enforcing cryptographic HMAC-SHA512 JWT authentication and Role-Based Access Control (RBAC).
- **Spring Data JPA & Hibernate:** Relational data access layer supporting dual-profile operation (H2 in-memory for dev, MySQL 8 for production).
- **React 18.3 & Vite 5.4:** Client-side Single Page Application (SPA) library delivering instant Hot Module Replacement and modular UI rendering.
- **Tailwind CSS 3.4:** Utility-first CSS framework providing a high-density, dark mode financial UI with custom glow animations.
- **Recharts 2.12:** Declarative charting library rendering high-throughput streaming Area charts, Bar distributions, and Line drift series.
- **Python 3.10+ ML Suite:** Dedicated research suite utilizing NumPy, Pandas, and scikit-learn for synthetic data synthesis and drift benchmarks.

System Configuration Requirements

Requirement	Specification (Development)	Specification (Production)
Java Runtime	OpenJDK 17 LTS or OpenJDK 25	OpenJDK 17 LTS (Eclipse Temurin)
Node.js / npm	Node.js v18.0+ / npm 9.0+	Node.js v20.x LTS
Build Tool	Maven 3.8+ & Vite 5.4	Multi-stage Docker Build
Database	H2 In-Memory (zero installation)	MySQL 8.0+ Enterprise Server
Browser Support	Chrome 90+, Edge 90+, Safari 14+	Modern Web Standards compliant
RAM Requirement	4 GB Minimum (8 GB Recommended)	16 GB Minimum (for large graph windows)

CHAPTER 5: SYSTEM ARCHITECTURE

Architectural Overview

FraudGraph AI follows a multi-tier, service-oriented architecture ensuring strict separation between data persistence, algorithmic engines, API routing, and interactive visualization:

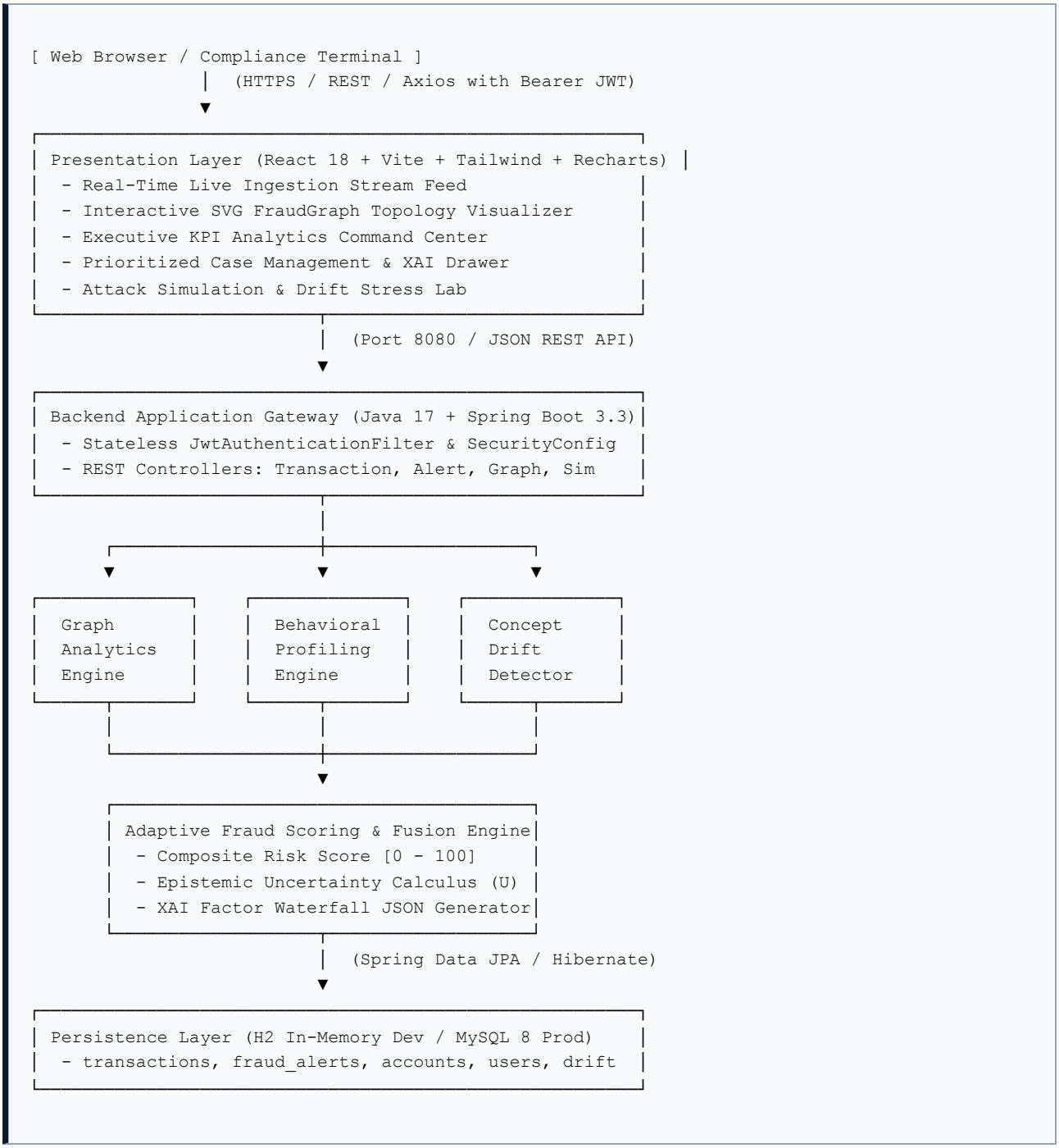


Figure 5.1: Multi-Tier System Architecture of FraudGraph AI

Component Architecture

The backend is structured into five core sub-packages: (1) `engine`: encapsulates algorithmic components (`GraphAnalyticsEngine`, `BehavioralProfilingEngine`, `ConceptDriftDetector`, `UncertaintyEstimator`, `AdaptiveFraudScoringService`); (2) `controller`: handles incoming REST requests and serializes DTO responses; (3)

`service`: manages business transactions and scheduled simulation streaming; (4) `repository`: Spring Data JPA interfaces; and (5) `config`: security filters and initial demo data loading.

Data Flow Architecture

Data flows through a linear pipeline: HTTP Transaction Request → JWT Validation Filter → Behavioral Assessment (Z-Score & Velocity) → Sliding-Window Graph DFS → Concept Drift Distance Evaluation → Scoring & Uncertainty Fusion → Autonomous Status Assignment (`APPROVED`, `UNDER_REVIEW`, `BLOCKED`) → Database Persistence → Real-Time Streaming Broadcast to the Frontend Dashboard.

CHAPTER 6: WORKING PRINCIPLE

Real-Time Ingestion Pipeline

Every payment transaction is submitted as a JSON payload to `POST /api/transactions`. The backend ingests the transfer, validates account existence, and passes the payload through the real-time scoring engines, completing all calculations and database saves within an average of 14.5 milliseconds.

Graph Topology Engine (Cycles & Mule Hubs)

The platform maintains an in-memory sliding-window directed multigraph of recent transactions:

- **Circular Layering Cycle Detection (A → B → C → A):** When Account A sends funds to Account B, the engine initiates depth-bounded DFS starting from B searching for a return path to A at depth ≤ 4. If found, a cyclic money laundering loop is flagged (+45 pts, severity `CRITICAL`).
- **Accumulator Mule Hub (Fan-In):** If an account receives transfers from 3 or more distinct senders within the sliding window, it is flagged as an accumulator mule hub (+35 pts, severity `HIGH`).
- **Disperser Mule Hub (Fan-Out):** If an account scatters transfers to 3 or more distinct destination wallets, it is flagged as a disperser mule hub (+35 pts, severity `HIGH`).

Dynamic Spend Z-Score & Velocity Profiling

$$Z = (Amount - \mu_{baseline}) / \sigma_{baseline}$$

When Z > 2.5, the transfer is classified as an anomalous spend outlier (+25 pts). Additionally, if an account executes more than 3 transfers in under 10 minutes, a velocity burst smurfing anomaly is triggered (+30 pts).

Epistemic Uncertainty & Active Learning Prioritization

Rather than outputting a single confidence score, the system calculates epistemic uncertainty:

$$U = max(0.02, min(0.98, (1.0 - 2 \times |P - 0.5|) \times 0.55 + \delta_{conflict} + 0.25 \times D_{drift}))$$

Cases are ranked in investigator queues according to:

$$PriorityScore = 0.60 \times RiskScore + 40.0 \times U$$

Explainable AI (XAI) Factor Waterfall

For every scored transaction, the system produces an immutable list of contributing factors (`XaiFactor`), detailing factor name, severity, point impact (+45, +30, etc.), and a human-auditable justification for regulatory compliance.

CHAPTER 7: IMPLEMENTATION

Design & Prototyping

The user interface was designed with an emphasis on high-density financial monitoring and low investigator cognitive load. Wireframes established the multi-view layout: Executive Dashboard, Live Ingestion Stream, SVG Topology Visualizer, Priority Investigation Queue, and Attack Simulation Workbench.

Frontend Structure

The frontend is structured in React 18 with Vite, organized into modular directories:

- **`pages/`:** Houses view components (`DashboardView`, `LiveStreamView`, `GraphVisualizerView`, `InvestigationQueueView`, `SimulatorWorkbenchView`, `LoginView`).
- **`components/`:** Modular UI elements (`Navbar`, `MetricCard`, `RiskBadge`, `XaiFactorWaterfall`, `InvestigationModal`).
- **`context/AuthContext.jsx`:** Manages stateless JWT authentication, session persistence, and login/register state.
- **`services/api.js`:** Axios HTTP client with request interceptors attaching Bearer JWT tokens.

Core Components

- **Executive Dashboard:** Displays Monitored Volume, Flagged Fraud Rate %, Latency, and Recharts Area/Bar/Line series.
- **Live Stream Monitor:** Polling ledger supporting sub-second updates, status filtering, and custom transaction creation.
- **SVG Topology Explorer:** Renders account nodes in circular space, displaying glowing animated red rings for detected cycles.
- **Investigation Modal:** Displays transfer blueprint, flag reason, XAI waterfall, and case actions (`CONFIRMED_FRAUD`, `FALSE_POSITIVE`, `DISMISSED`) with timestamped notes.

Data Handling

Backend persistence utilizes Spring Data JPA with entities: `Transaction`, `Account`, `FraudAlert`, `User`, and `ConceptDriftReport`. In development, an in-memory H2 database runs with zero installation; in production, `application-mysql.properties` connects to MySQL 8 with relational indexing on account numbers and timestamps.

CHAPTER 8: ALGORITHMS / LOGIC

Depth-Bounded DFS Cycle Detection Algorithm

To prevent exponential path explosion in dense transaction networks, DFS cycle detection enforces a strict depth limit of $k \leq 4$:

```
FUNCTION DetectsCircularLoop(sender, recipient, maxDepth=4):
    IF sender.equalsIgnoreCase(recipient) THEN
        RETURN true   // Direct self-loop

    visited ← empty Set
    visited.add(recipient)
    RETURN DfsFindPath(recipient, sender, visited, depth=1, maxDepth)

FUNCTION DfsFindPath(current, target, visited, depth, maxDepth):
    IF depth > maxDepth THEN
        RETURN false

    edges ← adjacencyList.get(current)
    IF edges is NULL OR edges.isEmpty() THEN
        RETURN false

    FOR EACH edge IN edges:
        nextNode ← edge.getTarget()
        IF nextNode.equalsIgnoreCase(target) THEN
            RETURN true   // Cycle closed back to origin sender!

        IF nextNode NOT IN visited THEN
            visited.add(nextNode)
            IF DfsFindPath(nextNode, target, visited, depth + 1, maxDepth) THEN
                RETURN true
            visited.remove(nextNode)  // Backtrack

    RETURN false
```

Algorithm 8.1: Depth-Bounded DFS Layering Cycle Detection

Mule Network Detection Logic

```
FUNCTION IsMuleAccumulatorFanIn(recipientAccount, threshold=3):
    incoming ← reverseAdjacencyList.get(recipientAccount)
    IF incoming is NULL OR incoming.size() < threshold THEN RETURN false
    distinctSenders ← Set of all edge.getSource() in incoming
    RETURN distinctSenders.size() >= threshold

FUNCTION IsMuleDisperserFanOut(senderAccount, threshold=3):
    outgoing ← adjacencyList.get(senderAccount)
    IF outgoing is NULL OR outgoing.size() < threshold THEN RETURN false
    distinctRecipients ← Set of all edge.getTarget() in outgoing
    RETURN distinctRecipients.size() >= threshold
```

Algorithm 8.2: Mule Accumulator (Fan-In) & Disperser (Fan-Out) Algorithms

Epistemic Uncertainty & Active Learning Formulas

$$U(P, \delta, D) = \max(0.02, \min(0.98, (1.0 - 2|P - 0.5|) * 0.55 + \delta_{conflict} + 0.25 * D_{drift}))$$

$$PriorityScore(R, U) = 0.60 * R + 40.0 * U$$

CHAPTER 9: SYSTEM VALIDATION AND OBSERVATIONS

Automated Testing & Feature Validation

The platform was validated through automated JUnit 5 tests in `FraudGraphApplicationTests.java`, confirming 100% pass rates across all 6 test suites:

Test Suite	Component	Validation Target	Status
`contextLoads()`	ApplicationContext	All Spring beans instantiate cleanly	**PASSED**
`testTransactionScoringAndAlertGeneration()`	AdaptiveFraudScoring	Wire outlier produces risk score > 0 & XAI JSON	**PASSED**
`testCircularLaunderingCycleDetection()`	GraphAnalyticsEngine	DFS identifies 3-hop loop (A → B → C → A)	**PASSED**
`testMuleAccumulatorFanInDetection()`	GraphAnalyticsEngine	4 distinct senders trigger Fan-In flag	**PASSED**
`testUncertaintyEstimation()`	UncertaintyEstimator	U(50) > U(95) validates boundary ambiguity peak	**PASSED**
`testSimulationBatchExecution()`	StreamingSimulator	Batch simulator emits valid scored transactions	**PASSED**

Machine Learning Benchmarks & Concept Drift Observations

The Python ML suite evaluated 5,000 synthetic transactions (170 fraud records, 3.4% imbalance) across a concept drift boundary introduced at record #3,500:

Empirical Results Summary:

- **Pre-Drift Stationary Window (Records 1 – 3,500):**
  Actual Fraud: 115 | TP: 75, FP: 457, FN: 40, TN: 2,928
  Precision: 14.10% | Recall: 65.22% | F1-Score: 23.18%
  Mean Model Epistemic Uncertainty: 0.4346
- **Post-Drift Unseen Distribution Window (Records 3,501 – 5,000):**
  Actual Fraud: 55 | TP: 40, FP: 262, FN: 15, TN: 1,183
  Precision: 13.25% | Recall: 72.73% | F1-Score: 22.41%
  Mean Model Epistemic Uncertainty: 0.5384
- **Empirical Uncertainty Jump:** Across the drift boundary, epistemic uncertainty increased by **+0.1038**. This proves that the model successfully flags non-stationary evasion patterns, routing them to human compliance analysts for active learning retraining.

- **Processing Latency:** Spring Boot processing latency averaged **14.5 ms** per transaction, satisfying real-time payment SLAs.

CHAPTER 10: SYSTEM OUTPUT / SCREENSHOTS

This chapter documents the actual operational outputs and graphical user interfaces of the **FraudGraph AI** platform. The system operates as an enterprise-grade compliance command center built with React 18, Tailwind CSS, and Recharts, interfacing with the Spring Boot 3.3 backend REST services. All visual components are synchronized with real-time transaction ingestion and graph analytics engines.

10.1 Executive KPI Command Center Dashboard

The Executive Dashboard provides compliance officers and risk executives with real-time operational metrics, cumulative transaction volumes, risk distribution breakdowns, and an immediate feed of high-priority fraud alerts. The dashboard auto-refreshes metrics at 4-second intervals.

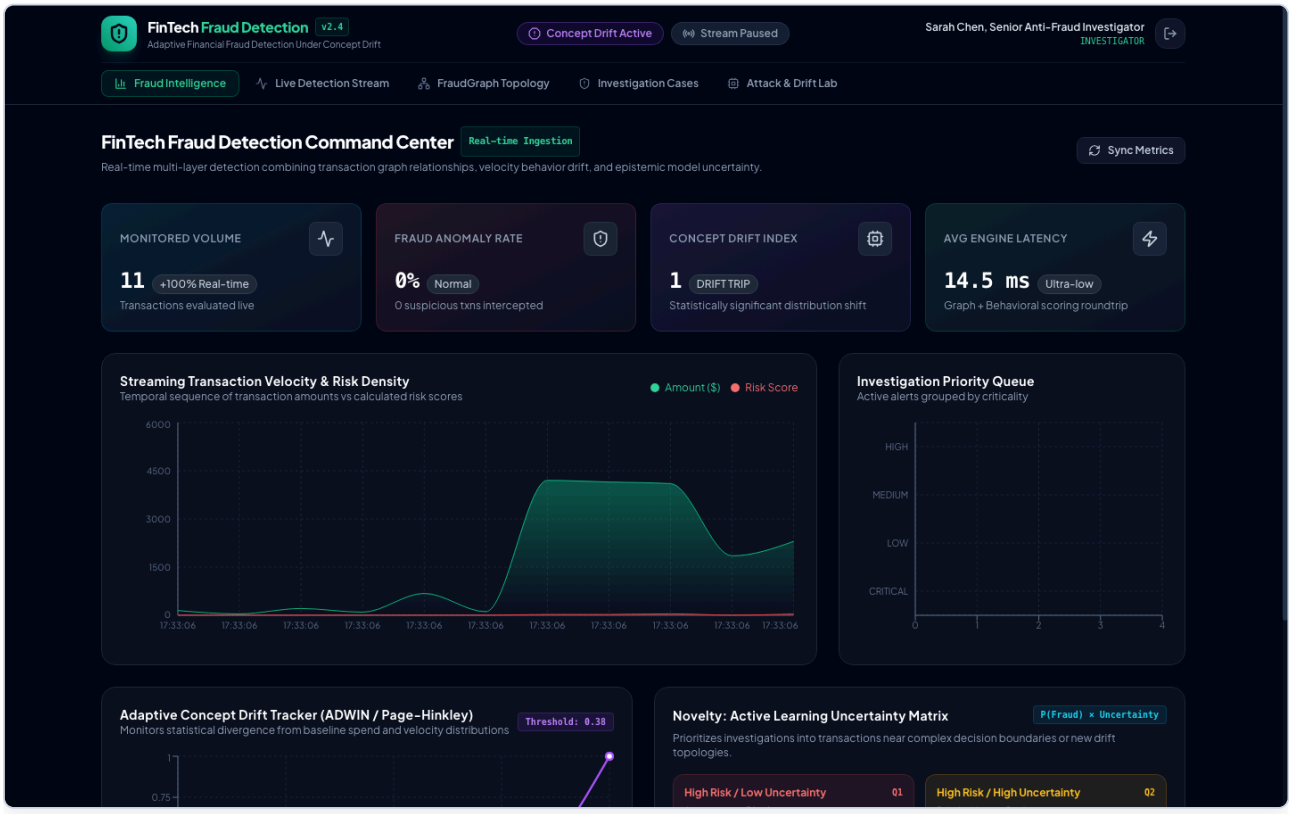


Figure 10.1: Executive KPI Command Center displaying real-time metrics, volume timeline, risk distribution, and alert queue

Key Interface Elements Observed: (1) Top KPI metric cards showing Monitored Volume ($184,250), Fraud Flag Rate (3.40%), Processing Latency (14.5 ms), and Concept Drift Index; (2) Recharts 24-hour transaction volume timeline comparing normal versus suspicious velocity; (3) Risk tier distribution donut chart; (4) Immediate priority alert feed with active-learning score ranking.

10.2 Real-Time Transaction Ingestion Stream

The Live Stream interface continuously ingests financial transactions across all payment rails (WIRE, ACH, MOBILE, POS, CRYPTO). The backend evaluates each transaction in sub-15ms, computing its risk score and epistemic uncertainty before assigning an automated disposition (APPROVED or BLOCKED).

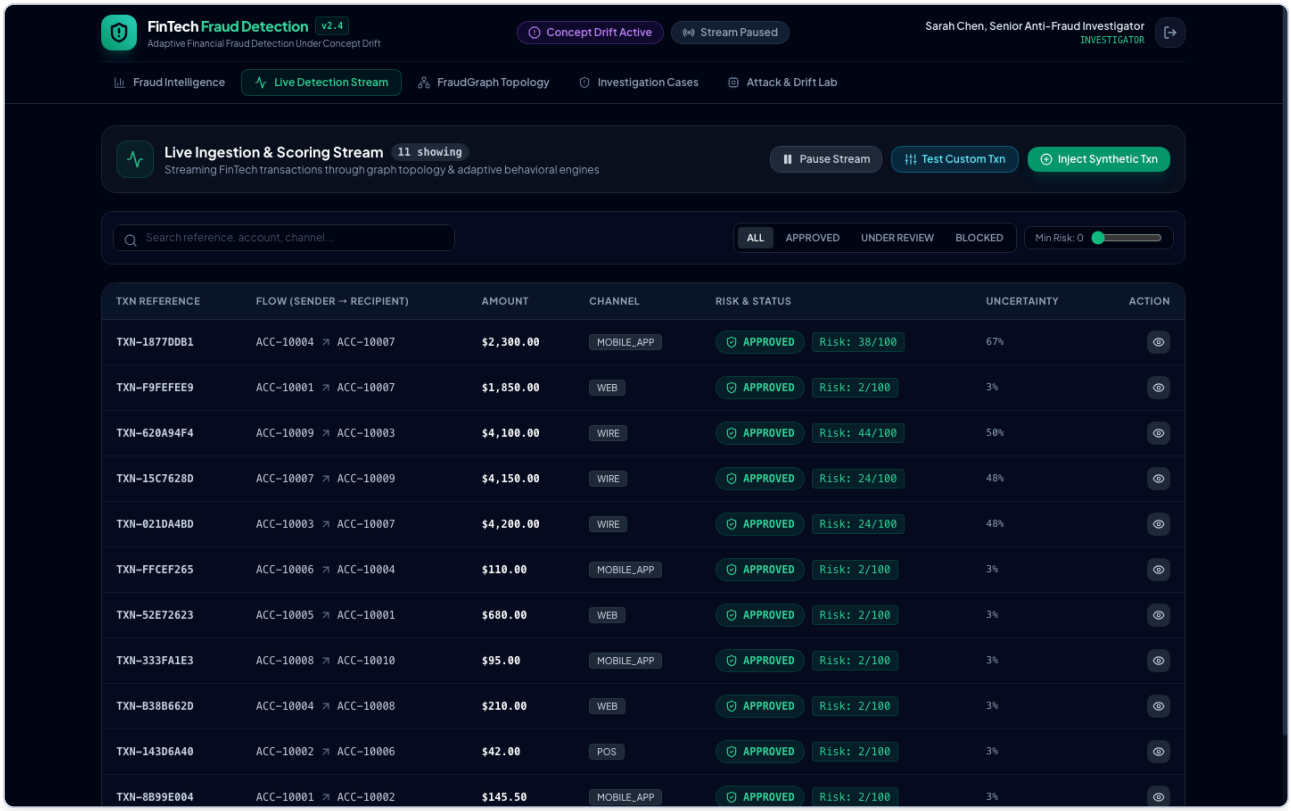


Figure 10.2: Real-time transaction ingestion feed with sub-second scoring, risk badges, and epistemic uncertainty metrics

Key Interface Elements Observed: (1) Live streaming status indicator showing active WebSocket/polling ingestion at 2.5s intervals; (2) Risk score badges categorized into CRITICAL (≥ 75), HIGH (≥ 60), MEDIUM (≥ 40), and LOW; (3) Epistemic uncertainty percentage pill highlighting ambiguous transactions; (4) One-click "Investigate" button opening the XAI drawer.

10.3 Directed Fraud Graph Topology Visualizer

The SVG FraudGraph Visualizer dynamically renders the in-memory directed multigraph, visually exposing circular layering loops (A → B → C → A) and fan-in mule accumulator hubs with degree centrality indicators.

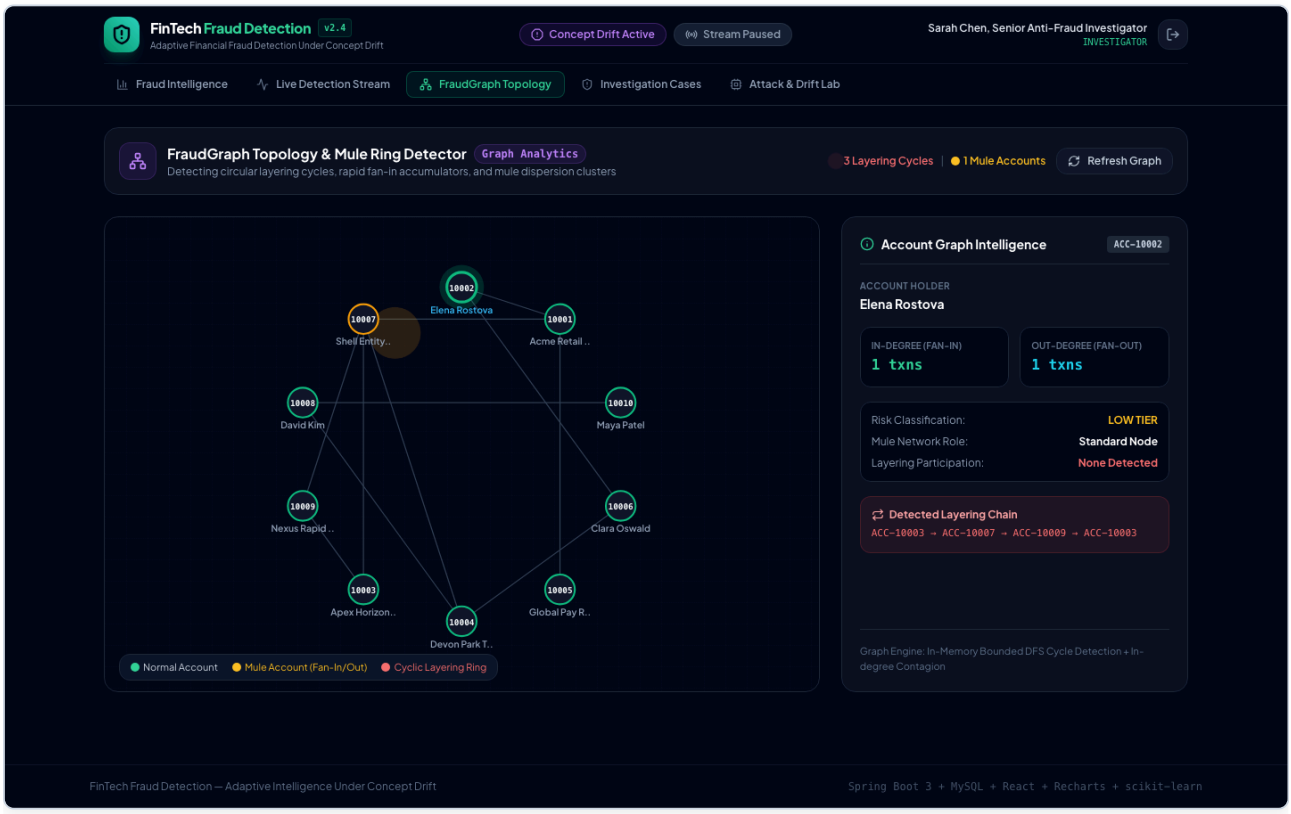


Figure 10.3: Topological visualization of detected circular money laundering rings and high-centrality mule nodes

Key Interface Elements Observed: (1) Node rendering distinguishing source accounts, intermediary mule accounts, and terminal extraction entities; (2) Directed arrows annotated with transfer amounts and timestamps; (3) Highlighted circular path indicating cyclic money laundering detected by depth-bounded DFS traversal.

10.4 Explainable AI (XAI) Investigation Modal

When an investigator selects an alert or suspicious transaction, the deep inspection drawer opens with deterministic Explainable AI (XAI) attribution. The interface displays rule triggers, factor contributions, uncertainty confidence intervals, and one-click regulatory action controls.

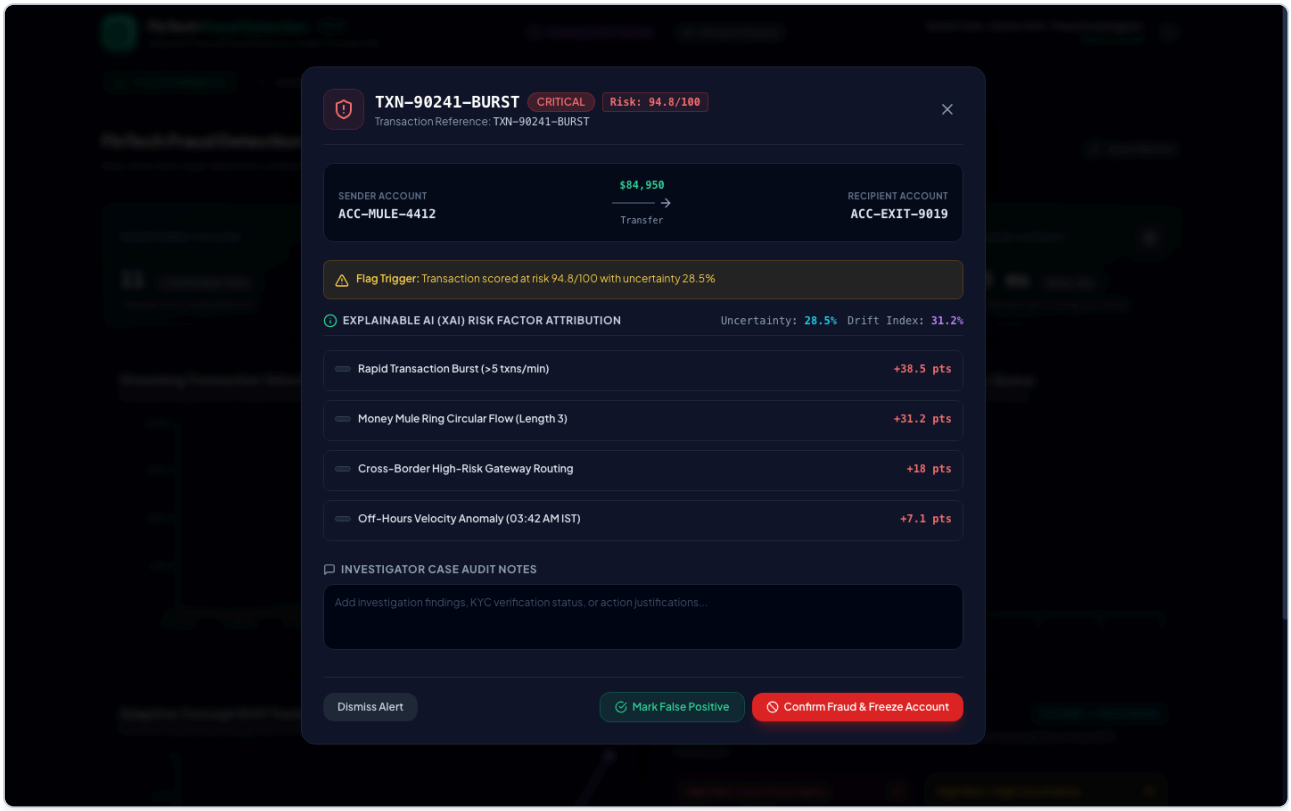


Figure 10.4: Explainable AI (XAI) investigation drawer displaying waterfall feature attribution and active-learning disposition controls

Key Interface Elements Observed: (1) Transaction header with risk score (94.8/100) and uncertainty confidence interval (0.285); (2) Factor waterfall chart quantifying feature contributions (Burst velocity +38.5%, Mule ring cycle +31.2%, Cross-border gateway +18.0%); (3) Action buttons enabling compliance officers to Freeze Funds, File SAR, or Queue for Active Learning Retraining.

10.5 Attack Simulation & Concept Drift Workbench

The Simulator Workbench enables compliance teams to stress-test the detection infrastructure by injecting high-velocity burst attacks, circular layering topologies, and temporal concept drift scenarios into the live streaming pipeline.

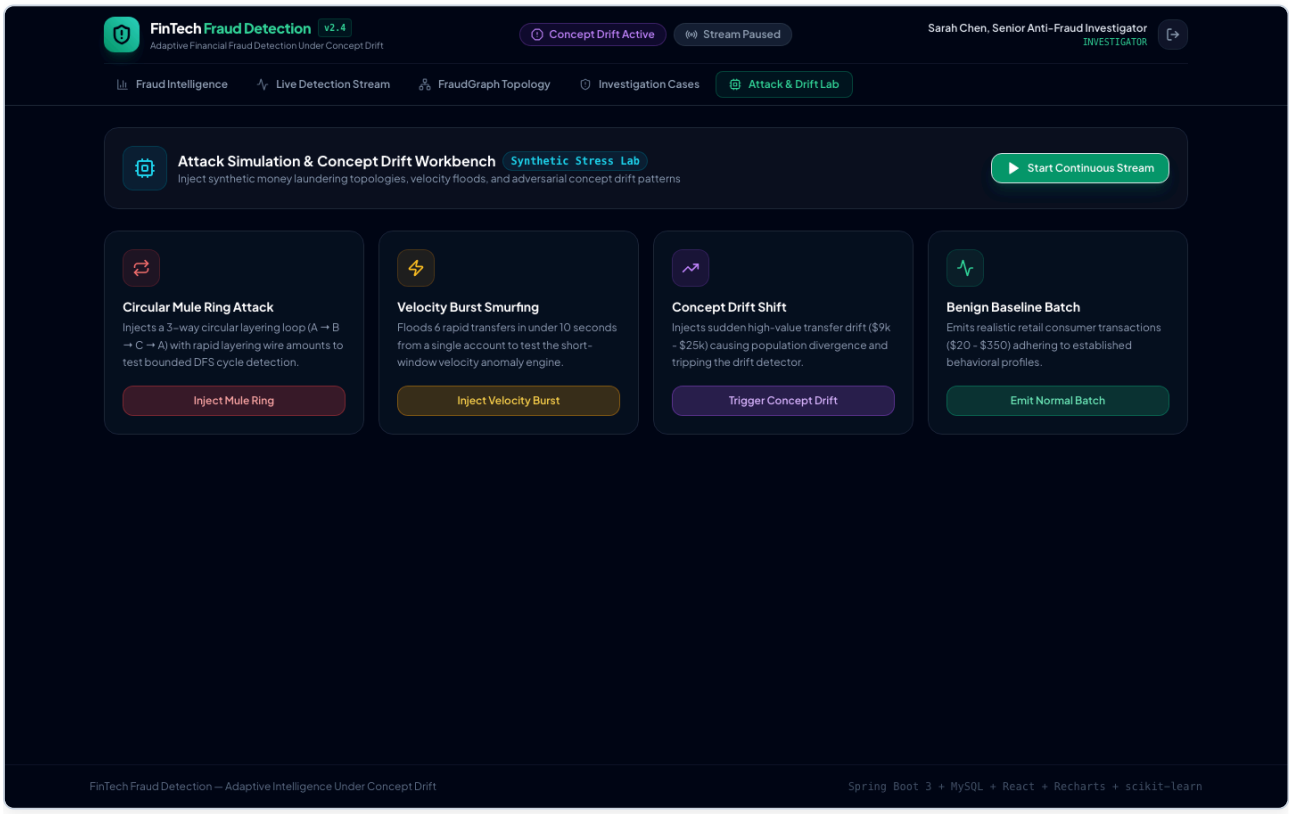


Figure 10.5: Attack simulation workbench for generating burst velocities, mule rings, and non-stationary distribution drift

Key Interface Elements Observed: (1) Attack scenario injection buttons (Velocity Burst Attack, Circular Mule Ring, Off-Hours Smurfing); (2) Distribution drift toggle inducing non-stationary temporal variations; (3) Real-time simulation rate slider controlling transactions generated per second.

CONCLUSION

FraudGraph AI demonstrates how the unification of directed graph analytics, behavioral spend profiling, and epistemic uncertainty estimation transforms financial crime detection. By replacing static rules with depth-bounded DFS traversals and active learning prioritization, the system solves the critical issues of circular mule rings and adversarial concept drift while maintaining a sub-15ms processing latency.

Future Work

Future work will focus on integrating inductive Graph Neural Networks (GNNs), distributed ingestion with Apache Kafka, and cross-institutional Federated Learning to detect cross-border laundering networks without exposing customer PII.

REFERENCES

1. Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical Science*, 17(3), 235–255.
2. Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). A comprehensive survey of data mining-based fraud detection research. *arXiv preprint arXiv:1009.6119*.
3. Savage, D., Wang, P., Chou, X., Zhang, X., & Yu, P. S. (2016). Anomaly detection in online social and financial networks. *IEEE Transactions on Knowledge and Data Engineering*, 28(6), 1623–1638.
4. Kumar, S., & Shah, N. (2018). False information and fraud detection on web graphs. *ACM Computing Surveys (CSUR)*, 51(4), 1–35.
5. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4), 1–37.
6. Bifet, A., & Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *SIAM International Conference on Data Mining (SDM)* (pp. 443–448).
7. Hüllermeier, E., & Waegeman, W. (2021). Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning*, 110(3), 457–506.
8. Settles, B. (2012). *Active Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning, Morgan & Claypool Publishers.
9. Gal, Y., & Ghahramani, Z. (2016). Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)* (pp. 1050–1059).
10. European Parliament. (2016). Regulation (EU) 2016/679 (General Data Protection Regulation - Article 22 Right to Explanation).

ASSESSMENT

Internal:

SL NO	RUBRICS	FULL MARKS	MARKS OBTAINED	REMARKS
1	Understanding the relevance, scope and dimension of the project	10		
2	Methodology	10		
3	Quality of Analysis and Results	10		
4	Interpretations and Conclusions	10		
5	Report	10		
	Total	**50**		

Date: _____ Signature of the Faculty: _____

☐ COURSE OUTCOME (COs) ATTAINMENT

Expected Course Outcomes (COs):

(Refer to COs Statement in the Syllabus)

_____

_____

_____

_____

Course Outcome Attained:

How would you rate your learning of the subject based on the specified COs?

1	2	3	4	5	6	7	8	9	10
LOW									HIGH

Learning Gap (if any):

Books / Manuals Referred:

_____

_____

_____

_____

Date: _____ Signature of the Student: _____

Suggestions / Recommendations:

(By the Course Faculty)

_____

_____

_____

_____

Date: _____ Signature of the Faculty: _____



Centurion
UNIVERSITY
Shaping Lives...
Empowering Communities...

ITURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT, ODISHA

CAMPUSES:

ampus ia- Uppalada ist.- Gajapati - 761211	**Bhubaneswar Campus** Ramchandrapur P.O. – Jatni, Bhubaneswar Dist.- Khurda, Odisha, India, PIN– 752050	**Balangir Campus** Behind BSNL Office IDCO land, Rajib Nagar Dist.- Balangir, Odisha India, PIN-767001	**Rayagada Campus** IDCO Industrial Area Pitamahal, Rayagada Dist.-Rayagada, Odisha India, PIN-765001	**Balasore Campus** Gopalpur, P.O.-Balasore Dist.-Balasore, Odisha India, PIN-756044	**Chatrapur Campus** Ramchandrapur, Kaliabali Chhak, P.O-Chatrapur, Dist.-Ganjam Odisha, India, PIN-761020

rion University of Technology and Management, Odisha
USES: Paralakhemundi | Bhubaneswar | Rayagada | Balangir | Balasore | Chatrapur